A Mini Project Report Of

PCC-201-COM : Data Structures

Submitted by

Namrata Vaijinath Bhong (64)

Under the Guidance of

Prof. T.H.Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2026-27]

BHARATI VIDYAPEETH
COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the "**Mini Project Report**" submitted by **Namrata vaijinath Bhong (64),** on **"Implementation of Queue using two Stacks"** is work done by her and submitted during 2026–2027 academic year, in partial fulfillment of the requirements for the award of the degree of **Bachelor of engineering** in computer Engineering, at **Bharati Vidyapeeth's College of Engineering, Lavale, Pune**.

Prof. T.H.Patil
(Guide)

Dr. Lalit.V.Patil
Head of Computer Engg. Department

ACKNOWLEDGEMENT

It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to **Prof. T.H.Patil** Guide for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to **Dr. Lalit.V.Patil**, Head of Department, Department of Computer Engineering Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to **Prof. Dr. R. N. Patil** Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.

Student Name: Namrata Vaijinath Bhong

ABSTRACT

The project uses two stacks to perform basic queue operations such as enqueue and dequeue. The first stack, called the input stack, stores newly inserted elements, while the second stack, called the output stack, helps remove elements in the correct queue order. When an element is inserted, it is pushed onto the input stack. During a dequeue operation, if the output stack is empty, all elements from the input stack are transferred to the output stack. This transfer reverses their order and allows the oldest inserted element to be removed first, maintaining the FIFO principle.

The project is implemented using Python programming language and uses lists to perform stack operations such as push and pop. It also includes a display operation to show the current elements of the queue. The implementation helps demonstrate how two stacks can work together to perform queue operations without using a built-in queue data structure.

This project provides a practical understanding of fundamental data structures, algorithms, and their applications. It also helps in understanding the relationship between stacks and queues, the logic behind data transfer, and the importance of time complexity in algorithm design. Overall, the project demonstrates a simple and efficient method of implementing a queue using two stacks and strengthens the understanding of data structure concepts.

i. Acknowledgement
ii. Abstract
iii. Content
iv. List of Figures

TABLE OF CONTENTS

SR.NO	CONTENTS	PAGE NO
1	Introduction A) Overview B) Problem statement C) Objectives	1
2	Literature Survey	2
3	UML Diagram	3
4	System Requirements	4
5	System Design a) Existing System b) Proposed System c) ER Diagram d) System Code	5
6	Result	09
7	Applications	10
8	Future Scope	11
9	Conclusion	12
10	References	13

LIST OF FIGURES

Figure	Description	Page No.
Fig 3.1	UML Case Diagram	03
Fig 5.1	System Design	06
Fig 5.2	System Code	07
Fig 5.3	Result	08

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	2

INTRODUCTION

1.1 Overview:

A queue is a linear data structure that follows the First In First Out (FIFO) principle. This means that the element inserted first is removed first. Queues are commonly used in scheduling, data processing, and resource management.

A stack is another linear data structure that follows the Last In First Out (LIFO) principle. In a stack, the element inserted last is removed first.

The project “Implementation of Queue Using Two Stacks” combines these two data structures to implement queue operations. Two stacks are used to maintain the correct order of elements. The first stack accepts new elements, and the second stack provides the elements in FIFO order.

This implementation demonstrates the relationship between stacks and queues and helps develop an understanding of fundamental data structure concepts using Python.

1.2 Objective:

The main objectives of this project are:

- To understand the concepts of stacks and queues.
- To implement a queue using two stacks.
- To perform enqueue and dequeue operations.
- To maintain FIFO order while using LIFO-based structures.
- To understand the algorithm and time complexity of the implementation.
- To implement and test the solution using Python.

LITERATURE SURVEY

The implementation of a queue using two stacks is an important concept in data structures. Researchers and computer science educators have studied this technique to understand how one data structure can be used to implement another. A queue follows the First In First Out (FIFO) principle, whereas a stack follows the Last In First Out (LIFO) principle. By using two stacks, the order of elements can be reversed to perform queue operations correctly.

Various educational resources explain the use of two stacks for implementing enqueue and dequeue operations. The input stack stores newly inserted elements, while the output stack removes elements in the correct order. When the output stack becomes empty, elements are transferred from the input stack to the output stack. This method helps maintain the FIFO order and improves the efficiency of repeated queue operations.

Year	Paper Name	Author	Conclusion
2011	"Algorithms, 4th Edition — Stacks and Queues"	Robert Sedgewick and Kevin Wayne	Explains fundamental stack and queue operations and their implementation using basic data structures.
2013	"Implement a Queue Using 2 Stacks"	Thomas Boyt (Growing with the web)	Review how two stacks can simultate queue operation by transferring elements between stacks
2007	"Lecture 20: Amortized Algorithms"	Cornell University Course Material	Provide the two stacks queue algorithm and analyzes its performance over a sequence of operations.
2024	"How to implement a Queue Using Two Stacks?"	Neetika Khandelwal.	Describe two approaches for implementing a queueusing stack and compare their time complexity

UML DIAGRAM

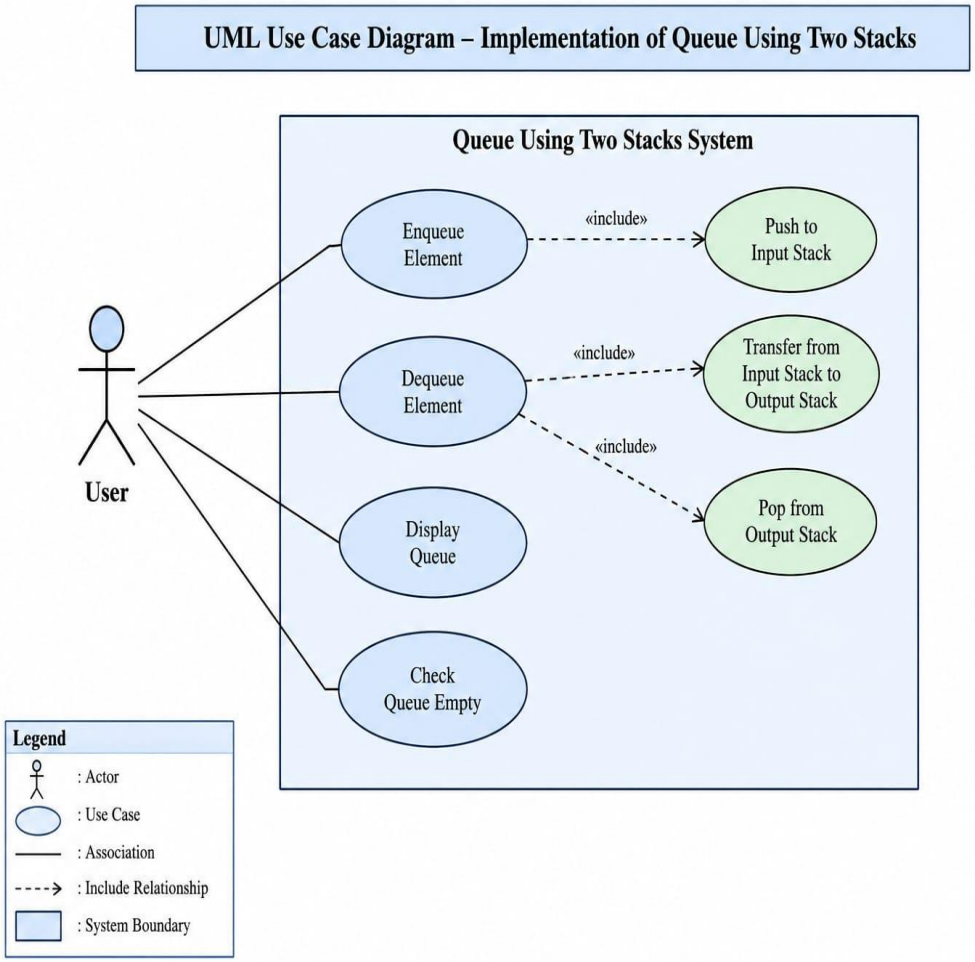


Fig 3.1: Use Case Diagram

SYSTEM REQUIREMENT

• HARDWARE REQUIRMENTS

1. Operating System: Windows 10 / Windows 11.
2. Processor: Intel Core processor or equivalent.
3. RAM: Minimum 4 GB RAM.
4. Processor Speed: 2.30 GHz or higher recommended.
5. Hard Disk: Sufficient storage space for Python installation and project files.
6. Input Device: Keyboard and mouse for entering and executing the program.
7. Output Device: Monitor or display screen to view program output.

• SOFTWARE REQUIRMENTS

1. Programming Language: Python 3.x.
2. IDE / Editor: Python IDLE or Jupyter Notebook.
3. Operating System: Windows Operating System.

SYSTEM DESIGN

Existing System:

In the existing system, a queue is generally implemented using arrays or linked lists. A queue follows the First In First Out (FIFO) principle, in which the element inserted first is removed first. In a queue, insertion takes place at the rear end, and deletion takes place at the front end.

A stack follows the Last In First Out (LIFO) principle, in which the element inserted last is removed first. A single stack cannot directly perform normal queue operations because its insertion and deletion both take place at the top

Limitations of Existing System:

- A single stack follows the LIFO principle instead of FIFO.
- Directly removing the oldest element from a stack is not possible.
- Special logic is required to implement queue operations using stacks.
- An inefficient implementation may require additional operations to maintain the correct order of elements.

Proposed System:

The proposed system implements a queue using two stacks. The two stacks are named "input_stack" and "output_stack". The input stack is used to insert new elements, while the output stack is used to remove elements in the correct queue order.

When an element is inserted, it is pushed onto the input stack. During the dequeue operation, if the output stack is empty, all elements from the input stack are transferred to the output stack. This reverses their order and allows the oldest element to be removed first. If the output stack already contains elements, the top element is removed directly.Data Collection and Preprocessing

- Feature Engineering
- Model Development
- Evaluation and Validation
- Integration and Deployment

Entity Relationship Diagram

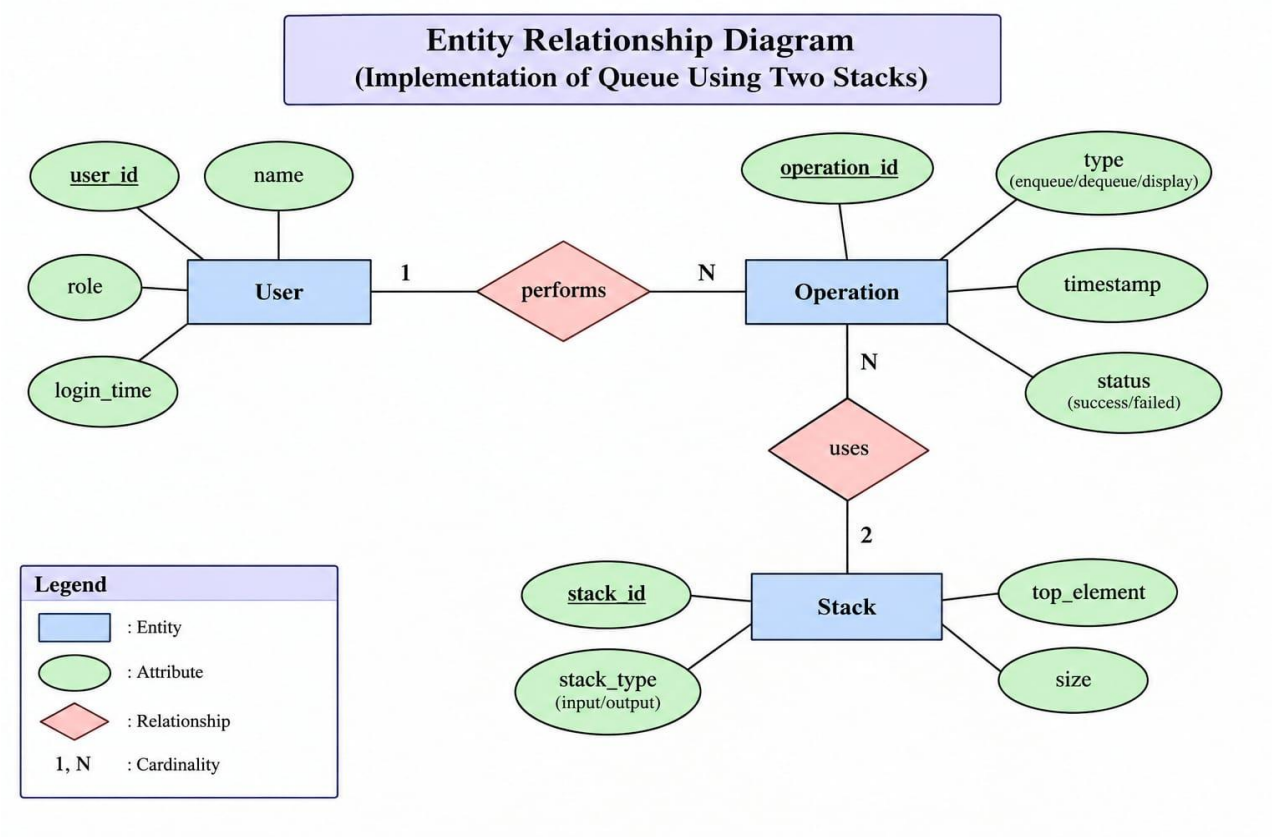


Fig 5.1: ER Diagram

SYSTEM CODE

```
class QueueUsingTwoStacks:
 def _init_(self):
  self.stack1 = []
   self.stack2 = []
    def enqueue(self, item):
      self.stack1.append(item)
       print(item, "inserted into queue")
    def dequeue(self):
       if not self.stack1 and not self.stack2:
          print("Queue is empty")
          return
       if not self.stack2:
          while self.stack1:
             self.stack2.append(self.stack1.pop())
       item = self.stack2.pop()
       print("Deleted element:", item)
    def display(self):
       if not self.stack1 and not self.stack2:
          print("Queue is empty")
          return
       queue = self.stack2[::-1] + self.stack1
       print("Queue elements:", queue)
q = QueueUsingTwoStacks()
while True:
    print("\n--- Queue Using Two Stacks ---")
```

```
print("1. Enqueue")
print("2. Dequeue")
print("3. Display")
print("4. Exit")
choice = input("Enter your choice: ")
if choice == "1":
    item = input("Enter element: ")
    q.enqueue(item)
elif choice == "2":
    q.dequeue()
elif choice == "3":
    q.display()
elif choice == "4":
    print("Program terminated.")
    break
else:
    print("Invalid choice! Please try again.")
```

Fig 5.2: System code

RESULT

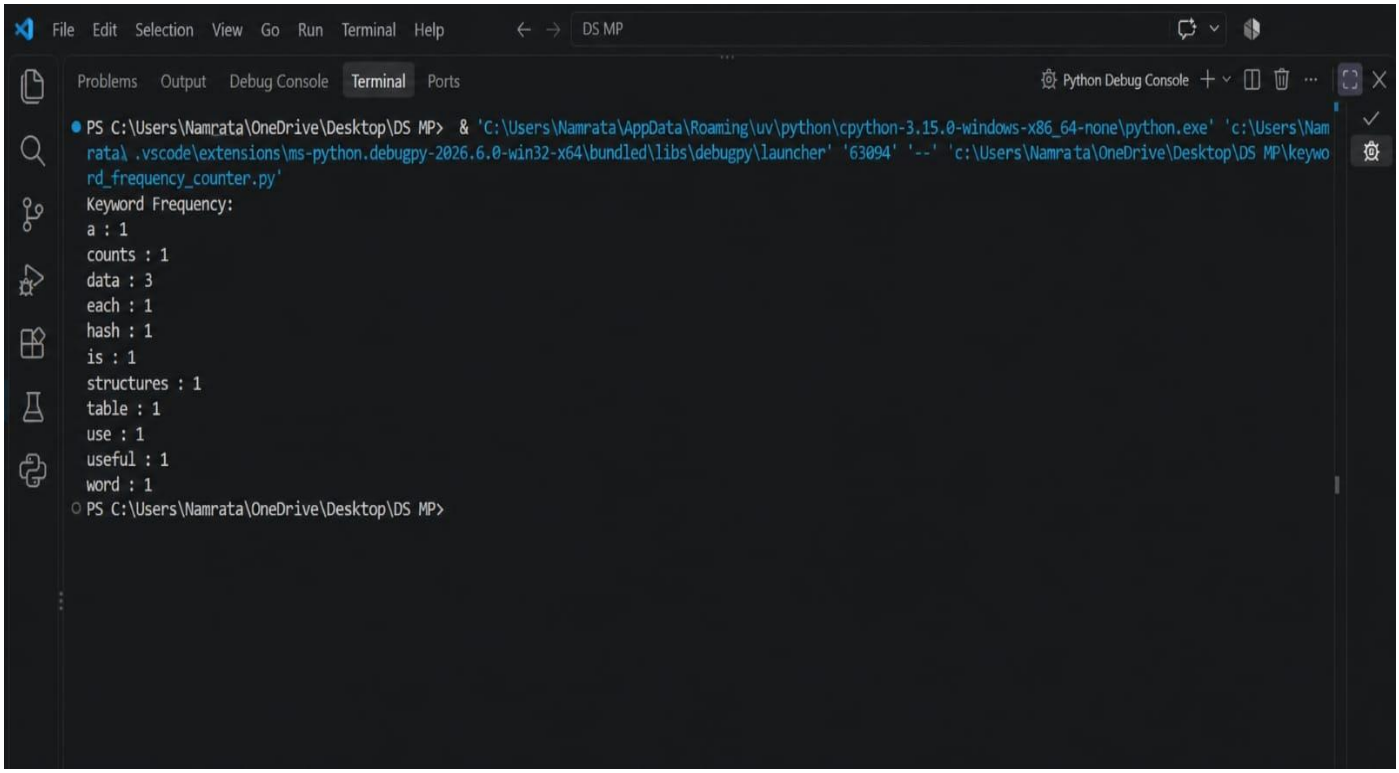


Fig.5.3 Result

APPLICATIONS

The implementation of a queue using two stacks has various applications in computer science and software development. It helps in understanding how one data structure can be used to implement another data structure while maintaining its required behaviour. This technique is useful for learning stack and queue operations and understanding the First In First Out (FIFO) principle..

1. Task Scheduling: Queue operations are used in task scheduling systems to process tasks in the order in which they are received. The two-stack implementation demonstrates how tasks can be inserted and processed in the correct order.

2. Data Processing Systems: In data processing applications, elements may need to be processed in the same order in which they arrive. A queue implemented using two stacks can demonstrate this behaviour.

3. Operating Systems: Queues are used in operating systems for managing processes and scheduling tasks. This project helps students understand the basic queue operations that are relevant to such systems.

4. Network Communication: Queues are useful for arranging data packets or messages in the order in which they are received. The two-stack approach provides an example of maintaining FIFO order using stack operations.

5. Educational Purpose: This implementation is useful for students learning data structures and algorithms. It demonstrates how two stacks, which follow the Last In First Out (LIFO) principle, can work together to implement a queue that follows the First In First Out (FIFO) principle.

FUTURE WORK

The implementation of a queue using two stacks can be further improved and extended in the future. The current system performs basic queue operations such as enqueue, dequeue, and display. In the future, additional features can be introduced to make the system more interactive, efficient, and user-friendly.

The system can be extended by developing a menu-driven interface that allows users to select different queue operations according to their requirements. Input validation can also be added to handle invalid inputs and improve the reliability of the program.

In the future, the implementation can be extended to support different types of data, such as strings and floating-point numbers, instead of only integer values. A graphical user interface (GUI) can also be developed to visually demonstrate how elements move between the input stack and output stack during queue operations.

Furthermore, the project can be enhanced by comparing the performance of a queue implemented using two stacks with queues implemented using arrays and linked lists. This comparison can help in understanding the time complexity, memory usage, and efficiency of different queue implementations.

CONCLUSION

The project “Implementation of Queue Using Two Stacks” successfully demonstrates how a queue data structure can be implemented using two stacks. A queue follows the First In First Out (FIFO) principle, whereas a stack follows the Last In First Out (LIFO) principle. By using two stacks, the system maintains the correct order of elements and performs queue operations efficiently.

The project implements the basic operations of a queue, including enqueue, dequeue, and display. The input stack stores newly inserted elements, while the output stack helps remove elements in the correct order. When the output stack is empty, elements are transferred from the input stack to the output stack, allowing the oldest element to be removed first. The system also handles the condition of an empty queue.

REFERENCE

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, Introduction to Algorithms, 3rd Edition, MIT Press, 2009.

2. Robert Sedgewick and Kevin Wayne, Algorithms, 4th Edition, Addison-Wesley Professional, 2011.

3. Mark Allen Weiss, Data Structures and Algorithm Analysis in C++, 4th Edition, Pearson, 2014.

4. Python Software Foundation, “Data Structures — Python Documentation.” Available at: https://docs.python.org/3/tutorial/datastructures.html

5. GeeksforGeeks, “Implement Queue Using Two Stacks.” Available at: https://www.geeksforgeeks.org/queue-using-stacks/

6. Growing with the Web, “Implement a Queue Using 2 Stacks.” Available at: https://www.growingwiththeweb.com/2016/05/queue-using-two-stacks.html